Data Structures Course Project Report

Mini Search Engine

Inverted-Index Construction, Query Processing, and Threshold Analysis

Project 8-1 Roll Your Own Mini Search Engine (C++17)

Main corpus: The Complete Works of William Shakespeare (195 documents)
Secondary corpus: IETF RFCs (3 documents)

Authors

Cen Ruiyang Deng Jun Lin Jiangtao

Date: 2026-10-28

Abstract

This project implements a mini search engine in C++17 for *The Complete Works of William Shakespeare*. It covers the four required tasks—word frequency statistics and stop-word identification, a stemmed inverted index, word and phrase queries (with a Boolean extension), and an experiment on query thresholds—and addresses the bonus question of 500,000 files and 400,000,000 distinct words with measured results and a clearly marked extrapolation.

The corpus is extracted from 1,211 HTML files mirrored from the MIT Shakespeare site and reduced to 195 documents (37 plays, 154 sonnets, and 4 long poems), with 152,632 lines and 930,911 words; the default tokenizer produces 866,626 body tokens. Three IETF RFCs form a second corpus that tests whether the engine is independent of the Shakespeare format.

For stop-word selection, a two-feature tf/df rule narrows the candidate band, and a length-normalized rate entropy separates words that are spread evenly across documents from words that are merely frequent. The resulting three-feature rule derives 39 stop words, the same count as the manually curated list, with an overlap of 35 words. The rule also adapts to the RFC corpus: it derives a different 58-word list there and correctly excludes the RFC 2119 keywords `must`, `may`, and `should` because their distributions are not flat enough.

For the index, Porter stemming maps 25,705 word forms to 14,947 stems. The formal index contains 14,937 terms, 117,704 postings, and 556,866 positions. The first persistence format (v1, uncompressed) occupies 3,685,655 bytes; the second format (v2, delta plus variable-length coding) reduces this to 1,725,694 bytes, or 46.82% of v1. Splay-tree, hash-table, and `std::map` dictionaries produce byte-identical indexes. A disk-based B+ tree sidecar (237,568 bytes) reduces cold-query I/O to 0.225% of a full load, and a SPIMI-style block build finishes in 17 runs and produces an index identical to the in-memory build.

The query program supports word, phrase, and Boolean queries and exposes four thresholds: `max-df`, `min-df`, `min-match`, and `top-x`. On a matrix of 15 queries and 201 measurements, the thresholds change query semantics rather than merely precision: removing a query term can turn a three-term conjunction into a single-term query and raise the result count from 30 to 90, while phrase queries ignore the term-side thresholds.

Correctness is supported by 18 CTest tests (8 Catch2 executables, 46 test cases, and 85,155 assertions) and by independent Python implementations that reproduce the word-count CSV, rebuild the full index, parse both persistence formats, and check 303 phrase queries. The bonus experiments use a synthetic corpus of 500,000 documents and 20,000,000 tokens: block construction needs 1.01 GiB of peak memory versus 2.00 GiB for the in-memory build, and both produce byte-identical indexes. The B+ tree dictionary is measured from 100,000 to 50,000,000 keys at a stable 15.100 bytes per key, with height 4 at 50,000,000 keys; extrapolating the measured fanout to 400,000,000 words gives about 6.04 GB on disk, height 4, and four page reads (about 16 KiB) per lookup.

Keywords: inverted index; word frequency; stop words; rate entropy; Porter stemming; splay tree; B+ tree; query thresholds; reproducibility

Contents

Abstract **1**

1 Introduction **4**
1.1 Problem Description 4
1.2 Purpose and Scope 4
1.3 Background 4
1.4 Corpus Collection 5
1.5 System Overview 5
1.6 Report Organization 6

2 Data Structure and Algorithm Specification **6**
2.1 Tokenization and Normalization 6
2.2 Word Frequency Statistics 7
2.3 Stop-Word Selection 8
2.4 Porter Stemming 8
2.5 Inverted Index 9
2.6 Dictionary Structures 10
2.7 Disk-Based B+ Tree 11
2.8 Block-Based External Construction 11
2.9 Query Processing and Threshold Design 12
2.10 Summary of Main Data Structures 13

3 Testing Results **14**
3.1 Test Environment and Method 14
3.2 Test Cases 14
3.3 Word Frequency and Stop-Word Results 16
3.4 Inverted Index and Query Results 19
3.5 Threshold Experiment Results 20
3.6 Bonus: Scalability Experiments 21
3.7 Summary of Testing Results 23

4 Analysis and Comments **24**
4.1 Time and Space Complexity 24
4.2 Comparison with Alternative Data Structures and Algorithms 24
4.3 Generalization: A Second Corpus 25
4.4 Limitations and Possible Improvements 26
4.5 Bonus Analysis: Scalability and Boundaries 27
4.6 Conclusions 27

References **28**

A Source Code **29**
A.1 Module Map 29
A.2 Comment Coverage 29

B Reproduction Commands and Key Outputs **30**
B.1 Environment, Build, and Tests 30
B.2 Tasks 1 and 2 30
B.3 Task 3, Threshold Experiments, and Independent Cross-Checks 30
B.4 Bonus: B+ Tree Scaling and the 500,000-Document Corpus 31

C Data Format Reference **32**
C.1 Corpus Manifest and Text Layer 32
C.2 Version 1 Index Layout and Version 2 Differences 32
C.3 Disk B+ Tree Sidecar 33

Author List **34**

Declaration **35**

Signatures **36**

1 Introduction

1.1 Problem Description

A search engine answers a simple question—which documents contain this word or phrase?—but the answer is only useful if it is cheap to obtain and trustworthy when checked. This project builds such an engine for *The Complete Works of William Shakespeare*. The course asks for four pieces of work: (1) count words and identify stop words; (2) build an inverted index with stemming that excludes the stop words from part (1); (3) implement a query program that accepts a word or phrase and returns document IDs; and (4) run experiments that show how thresholds on a query affect the result. The bonus question asks whether the design still works at the scale of 500,000 files and 400,000,000 distinct words.

The main difficulty is not any single algorithm but the coupling between the stages. Stop-word removal changes position numbering; stemming changes the term vocabulary; the query program must replay exactly the normalization used at build time. A stop word that is left out of the index cannot be found later, and a phrase that contains a stop word changes its meaning when that word is dropped. The report therefore treats the pipeline as one specification rather than four independent exercises.

1.2 Purpose and Scope

The purpose of this report is to document what was built, how it was tested, and where the design stops working. Chapter 2 specifies the data structures and algorithms, Chapter 3 reports the test cases and measured results, and Chapter 4 analyzes complexity, compares the chosen structures with alternatives, and lists the limitations. The report is deliberately evidence-oriented: every measured number comes from a rerun on 2026-10-20, and the commands that reproduce the runs are collected in Appendix B. No measurement in this report was invented for the write-up, and no number was recomputed from a different corpus or configuration.

The scope is the single-machine, batch-built engine described here. Distributed indexing, incremental updates, and Unicode tokenization are discussed as future work but are not claimed as implemented features.

1.3 Background

An *inverted index* maps each term to a posting list that records the documents containing the term and, in this project, the positions at which the term occurs [2]. A word query intersects the posting lists of its terms; a phrase query additionally checks that the positions are consecutive. The two basic statistics are the term frequency $\mathrm{tf}(t)$, the number of occurrences of term t in the corpus, and the document frequency $\mathrm{df}(t)$, the number of documents containing t. Their ratio separates a word that is frequent because it occurs many times in one document from a word that is spread across the collection.

Stop words are terms with little discriminative value, such as function words. There is no universal list: a word that is noise in one corpus can be content in another. This project derives the list from the corpus itself and checks the derivation on a second corpus. *Stemming* reduces inflected forms to a common stem so that, for example, `agree`, `agreed`, and `agreement` share an index entry [1]. Stemming improves recall but can over-conflate unrelated words, so the implementation is checked against an independent implementation on the full vocabulary.

The dictionary that maps terms to posting lists is itself a data structure problem. A balanced search tree, a hash table, a splay tree, and a disk-based B+ tree offer different trade-offs between lookup time, ordered traversal, and memory [3]. The query thresholds are a separate design axis: `max-df` and `min-df` filter query terms, `min-match` changes the Boolean condition, and `top-x` truncates the ranked output. The experiment in Chapter 3 shows that these controls are not interchangeable.

1.4 Corpus Collection

1.4.1 Corpus Source

The main corpus is *The Complete Works of William Shakespeare* from the MIT Shakespeare site [4] and its official GitHub mirror. The mirror contains three overlapping copies of the works: the current lowercase play directories, an older uppercase classification directory, and Moby plain text files. To avoid mixing editions, the site's own index page is used as the authoritative table of contents. This yields 195 documents: 37 plays, 154 sonnets, and 4 long poems, totaling 152,632 lines and 930,911 words in 4,919,371 bytes of source text.

A second corpus checks that the engine is not tied to Shakespeare. It contains three IETF RFCs—1035 (DNS), 5321 (SMTP), and 7231 (HTTP semantics)—stored as plain text: 3 documents, 9,946 lines, and 81,848 whitespace-separated words. The same C++ binary processes both corpora; only the manifest and the text layer change.

1.4.2 Preprocessing Overview

`tools/extract.py` converts each HTML page into a text layer with one semantic unit per line, and writes `manifest.tsv` as the document catalog and path map. Line markers identify headings such as `[ACT I]`, stage directions such as `(Exit Ghost)`, and speaker labels such as `@BERNARDO`; the tokenizer ignores these lines by default. The marker must match at both the first and the last character. A line that only starts with `[` but continues as dialogue is body text, not a heading. This rule is essential for the corpus counts: an earlier comparison script tested only the first character, treated lines such as `[To HELENA] The best wishes ...` as headings, and undercounted the token stream by 4,340 tokens.

The detailed token rules are part of the specification in Section 2.1; the corpus-level view is enough here. All tokens are lowercased. The default configuration counts body lines only and produces 866,626 tokens. The flags `--keep-headings`, `--keep-speakers`, and `--keep-stage` can retain each marker class when a different counting convention is needed.

1.4.3 Special Cases

Three corpus defects required explicit handling. First, the Moby edition writes 610 bracketed stage directions; 57 are whole-line forms such as `[Sings]`, and 553 are line-start fragments followed by dialogue, such as `[To HELENA] The best wishes ...`. The extractor relocates these to parenthesized stage lines. Eight mid-line bracketed forms are editorial insertions or missing-text markers and are left in place. The net effect is a reduction of 849 tokens, which matches the fragment counts exactly. Second, `Poetry/elegy.html` contains the malformed tag `</TITLE` without a closing angle bracket; the HTML parser never leaves the head and extracts no text. Treating the start of `<body>` as the end of the head fixes the file. Third, `2henryiv.html` contains a source artifact, `<h3>None</h3>`, which is discarded rather than turned into a word.

The text layer and the token stream are checked against an independent Python implementation, so a preprocessing error is visible as a per-document difference rather than as a plausible but wrong total. The individual test cases are listed in Section 3.2.

1.5 System Overview

Figure 1 shows the data flow. On the offline side, the HTML corpus becomes a text layer, the tokenizer produces a token stream, and the stream feeds two branches. The counting branch writes `wordcount.csv` and derives the stop-word list. The indexing branch applies the same stop-word filter and the optional Porter stemmer, inserts each term into the dictionary, and serializes the postings to `index.bin`; an optional B+ tree sidecar stores the term-to-posting mapping for lazy lookup. On the online side, the query text goes through the same tokenizer, stop-word filter, and stemmer, then looks up the dictionary, intersects posting lists, verifies positions for phrases, applies the thresholds, and returns document IDs and titles.

Keeping the two branches on one tokenizer is a deliberate constraint. It makes the stop-

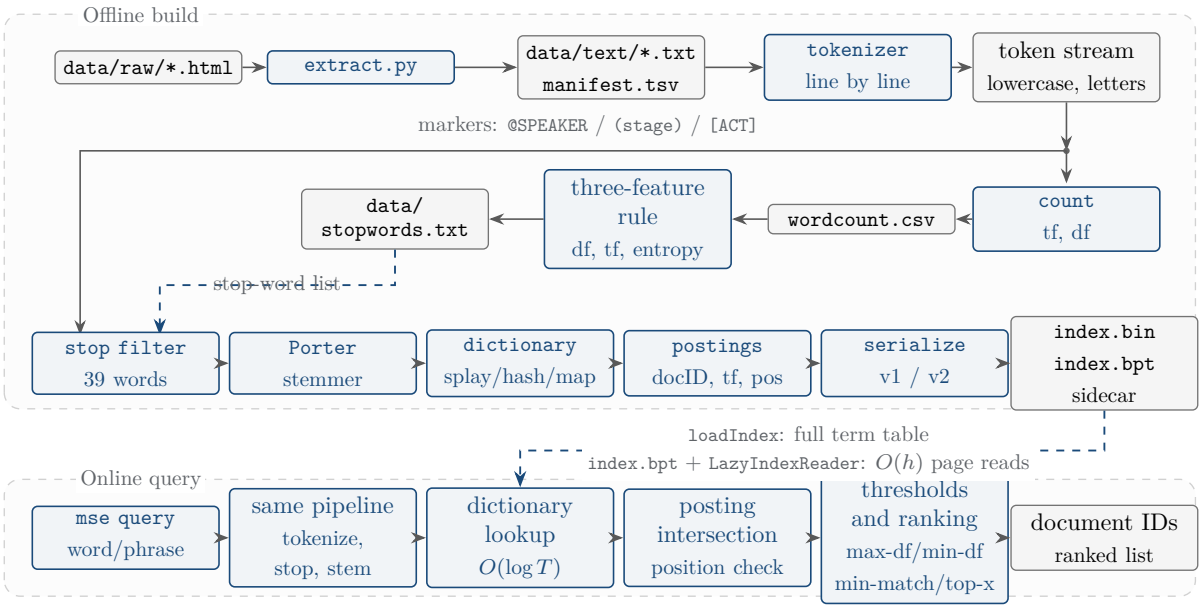


Figure 1: System data flow. The offline side extracts the HTML corpus into a text layer, tokenizes it, removes stop words, applies Porter stemming, and writes the inverted index; word-frequency counting and stop-word derivation form a bypass. The online side reuses the same normalization pipeline, then performs dictionary lookup, posting intersection and position verification, applies the four thresholds, and returns document IDs. The dashed path is the B+ tree sidecar written by `--dict-file`; a query reads $O(h)$ pages, where h is the tree height.

word list, the stemmer, and the query program share a single definition of a term, and it lets the index header record the processing flags so that a query cannot silently use a different normalization.

1.6 Report Organization

Chapter 2 gives the data-structure and algorithm specification, including pseudo-code for the main operations. Chapter 3 presents the test environment, the required test-case table, and the measured results for the four tasks and the bonus experiments. Chapter 4 analyzes time and space complexity, compares the chosen structures with alternatives, generalizes to the RFC corpus, and lists limitations and improvements. Appendix A records the source-code organization and comment coverage, Appendix B gives the reproduction commands, and Appendix C documents the data formats. The Author List, Declaration, and Signatures follow the appendices.

2 Data Structure and Algorithm Specification

This chapter specifies the data structures and algorithms used by the engine. The order follows the data flow: tokenization, counting, stop-word selection, stemming, index construction, dictionary structures, disk-based storage, external construction, and query processing. Pseudo-code is written in a language-independent form; identifiers such as `lastDoc` and `posStart` match the implementation.

2.1 Tokenization and Normalization

The corpus is stored as a *text layer*: one semantic unit per line, with a line marker that identifies headings, stage directions, and speaker labels. `manifest.tsv` is the catalog; each row contains the document ID, title, text path, and source path, and an optional `format` column selects `marked` (default) or `plain` (used by the RFC corpus).

A token is a maximal run of ASCII letters that may contain internal apostrophes. All tokens are lowercased. The apostrophe rules are:

- an internal apostrophe is kept: `don't` and `o'er` are single tokens;
- a leading apostrophe is dropped: `'Tis` becomes `tis`;
- a trailing apostrophe acts as a separator: `friends'` becomes `friends`;
- hyphens, digits, and ordinary punctuation are separators: `to-day` becomes `to` and `day`, and `1660s` becomes `s`.

Marker lines are skipped by default. A heading such as `[ACT I]`, a stage direction such as `(Exit Ghost)`, or a speaker label such as `@BERNARDO` is recognized only when the first and last characters match the expected pair. A line that begins with `[` but continues as dialogue is body text. The optional flags `--keep-headings`, `--keep-speakers`, and `--keep-stage` retain each class for experiments that need a different counting convention.

```
for each line L in the text layer:
    if L is a marker line and the marker class is not kept:
        continue
    for each maximal run T of ASCII letters and internal apostrophes:
        T = drop leading apostrophe; T = lowercase(T)
        emit T
```

Normalization is the composition of three stages: tokenize, optionally remove stop words, and optionally apply the Porter stemmer. The index header stores two flags for the latter two stages, and the query program reads these flags and replays the same composition. This is the mechanism that keeps a query from using a different definition of a term than the index.

2.2 Word Frequency Statistics

2.2.1 Design Principles

For a term t, the engine reports its term frequency and document frequency:

$$\mathrm{tf}(t) = \sum_{d} \mathrm{tf}(t, d), \qquad \mathrm{df}(t) = |\{d : \mathrm{tf}(t, d) > 0\}|. \tag{1}$$

It also computes the document-frequency ratio $\mathrm{df_ratio}(t) = \mathrm{df}(t)/195$ and the term-frequency share $\mathrm{tf_share}(t) = \mathrm{tf}(t)/866\,626$. The first measures coverage; the second measures volume. Looking at only one of them makes a dialogue word diluted by the 154 short poems look rare when it is actually common.

The counter reuses the formal normalization pipeline. Four configurations are supported: the default (no stop words, no stemming), stop words only, stemming only, and the formal configuration (both). The same order—tokenize, filter, stem—is used by the index and query programs.

2.2.2 Algorithm

Documents are scanned in manifest order. Each term row stores its accumulated frequency and the ID of the last document in which it was seen. The document frequency is incremented only when the current document ID differs from `lastDoc`; repeated occurrences inside one document therefore contribute once to df.

```
for each document d in manifest order:
    for each token t in tokenize(d):
        row = table[t]
        row.tf += 1
        if row.lastDoc != d.id:
            row.df += 1
            row.lastDoc = d.id
```

With a hash table the expected time is $O(N)$ for N tokens, plus $O(T \log T)$ for sorting the output rows; with a splay dictionary it is $O(N \log T)$ amortized. The memory bound is $O(T)$ rows. The four CSV outputs are deterministic: the same input and flags produce the same bytes, and the files contain no timestamps.

2.3 Stop-Word Selection

2.3.1 The tf/df Candidate Rule

The simplest candidate rule asks a word to be both widespread and frequent:

$$\text{df_ratio}(t) \geq 0.75 \quad \wedge \quad \text{tf_share}(t) \geq 0.001. \tag{2}$$

It returns only nine words. The reason is the corpus composition: 154 short poems account for 79% of the documents, and many function words do not appear in every poem, so their document-frequency ratios are diluted. For example, `a` has a ratio of 0.7077, just below 0.75. Lowering the threshold to 0.40 produces 48 candidates, but content words such as `love` and `time` enter the band as well.

2.3.2 Length-Normalized Rate Entropy

To separate "spread evenly across many documents" from "merely frequent," the engine computes a length-normalized rate entropy. Let L_d be the number of kept tokens in document d:

$$r_{t,d} = \frac{\text{tf}(t,d)}{L_d}, \qquad q_{t,d} = \frac{r_{t,d}}{\sum_{d'} r_{t,d'}}, \qquad \widehat{H}(t) = \frac{-\sum_d q_{t,d} \log_2 q_{t,d}}{\log_2 \text{df}(t)}. \tag{3}$$

After length normalization, function words sit above 0.95, content words such as `love`, `king`, and `heart` are visibly lower, and a proper noun with df = 1 has entropy zero by definition. The denominator $\log_2 \text{df}(t)$ makes the statistic comparable across words that occur in different numbers of documents.

2.3.3 The Three-Feature Rule

The final rule requires a word to be wide, heavy, and flat:

$$\text{df_ratio} \geq r \quad \wedge \quad \text{tf_share} \geq s \quad \wedge \quad \widehat{H} \geq h. \tag{4}$$

The default $(r, s, h) = (0.75, 0.001, 0.93)$ yields nine words. The experimental configuration $(0.40, 0.001, 0.93)$ yields 39 words and overlaps the manually curated list in 35 words. The automatic list adds `hath`, `o`, `thee`, and `thou`, and omits `more`, `or`, `than`, and `their`; each difference can be explained by the rate entropy.

```
for each term t in the vocabulary:
    if df_ratio[t] >= r and tf_share[t] >= s
       and rate_entropy[t] >= h:
        emit t
```

The rule is deterministic and uses no external stop-word list. Changing a threshold changes only the strictness of the band, not the structure of the rule.

2.4 Porter Stemming

2.4.1 Comparison of Public Stemmers

Four public descriptions were compared: the original Porter (1980) paper, the canonical C implementation on the author's site, the NLTK `PorterStemmer` in its three compatibility modes, and the Snowball Porter2 algorithm [1, 5, 6, 7]. The implementation follows the 1980 paper plus the three marked `--DEPARTURE--` changes in the canonical C code: words of two letters or fewer are left unchanged, `bli` is changed to `ble`, and `logi` is changed to `log`. This is the same behavior as NLTK's `MARTIN_EXTENSIONS` mode, which the full-vocabulary cross-check uses as an oracle.

2.4.2 Algorithm

Before the Porter rules, contractions are reduced: the longest matching suffix among `n't`, `'s`, `'d`, `'ll`, `'re`, `'ve`, `'t`, and `'m` is removed, and remaining apostrophes are deleted. Thus `o'er` becomes `oer`, `marry'd` becomes `marryd`, and the irregular forms keep their rule result, for example `can't` becomes `ca` and `won't` becomes `wo`.

The main flow executes steps 1a, 1b, 1c, 2, 3, 4, 5a, and 5b. The core condition is the measure m (the number of vowel-consonant sequences) and the predicates `*v*`, `*d`, and `*o`. The implementation follows "the first matching suffix decides": once a suffix matches, only that rule's condition is checked; if the condition fails, the step stops and does not fall back to a shorter suffix. This detail matters: an earlier version continued to shorter suffixes after a failed condition, over-stemmed 41 word forms (for example, `agreement` became `agreem`), and was caught by the full-vocabulary differential test.

```
stem(word):
    word = strip_longest_contraction(word)
    for step in (1a, 1b, 1c, 2, 3, 4, 5a, 5b):
        for suffix in step.suffixes_by_length:
            if word ends with suffix:
                if step.condition(word, suffix):
                    word = step.replace(word, suffix)
                break          # first match wins; do not try shorter suffixes
    return word
```

The worst-case cost is $O(L^2)$ for a word of length L because suffix matching and condition checks rescan the word; in practice L is small and the observed throughput is dominated by the dictionary, not by the stemmer.

2.5 Inverted Index

2.5.1 Posting Lists and Invariants

Each term has a posting list ordered by document ID. A posting entry stores $(\text{docID}, \text{posStart}, \text{tf})$, where `posStart` is the start of the entry's positions in a separate position array and `tf` is the number of occurrences in that document. Positions are offsets in the stop-word-filtered token stream, so a phrase query can verify adjacency without returning to the original text.

The loader checks five structural invariants: document IDs are dense from 1 to N; document IDs within one posting list are strictly increasing; the `tf` of each entry equals its number of positions; the total number of positions equals the total number of kept tokens; and the term table is in strict lexicographic order. The same invariants are enforced by `validateIndex()` after loading.

2.5.2 Build Algorithm

The builder scans the document stream and appends postings incrementally; at the end, the dictionary is frozen into a sorted term table.

```
for each document D in manifest order:
    pos = 0
    for each token t in tokenize(D):
        if stop_words.contains(t): continue
        term = stem(t) if stem_flag else t
        dict.upsert(term).append(D.id, pos)
        pos += 1
    index.docs.push({D.id, D.title, pos})
index.terms = dict.sorted_entries()
```

The formal configuration produces 14,937 terms, 117,704 posting entries, 556,866 positions,

and a 3,685,655-byte v1 index. The file layout is shown in Figure 2.

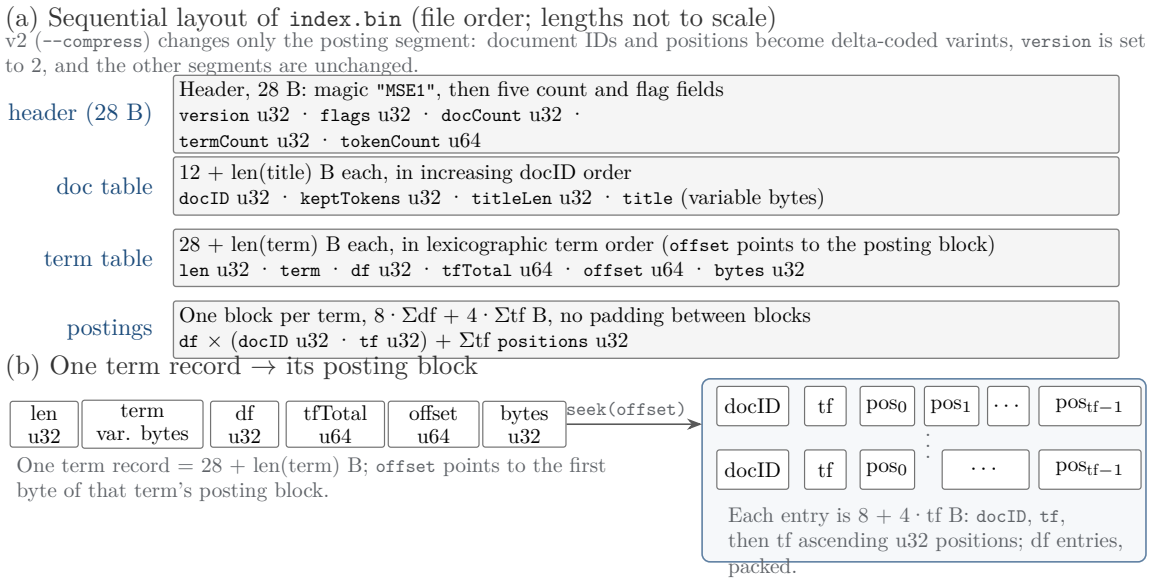


Figure 2: Index layout: `index.bin` stores the header, document table, term table, and posting segment in order. A term record points to its posting block with `offset` and `bytes`. Within a block, each posting stores the document ID and term frequency followed by tf ascending positions, so a phrase query only needs to compare adjacent positions.

2.5.3 Persistence Formats v1 and v2

Persistence is part of the index specification, not an afterthought. Format v1 writes the header, document table, term table, and posting blocks in order. All offsets are computed before writing, and the reader parses the same layout and replays the invariant checks. The same index object therefore always produces the same byte sequence.

Format v2 changes only the posting region: document IDs are stored as deltas, positions as deltas, and frequencies and deltas use a 1–4 byte byte-aligned variable-length code. The header, document table, and term table are unchanged, except that `version` is set to 2. On the formal index, v2 reduces 3,685,655 bytes to 1,725,694 bytes, or 46.82% of v1, with identical query output.

The variable-length code uses the top two bits of the first byte for the total length and stores 6, 14, 22, or 30 payload bits in 1, 2, 3, or 4 bytes, respectively. Values are little-endian. A document-ID delta is at least 1; a position delta is at least 0.

2.6 Dictionary Structures

The build-time dictionary exposes three operations: `upsert`, `find`, and `sortedEntries`. The primary implementation is a self-implemented splay tree; a hash table and `std::map` are alternative implementations used for comparison.

2.6.1 Splay Tree

The workload explains the choice. The index is insert-only, so the expensive deletion cases of a red-black tree never occur. The token stream is Zipf-distributed, and splaying an accessed node to the root keeps frequent terms near the top. At the end of the build, an in-order traversal produces the sorted term table directly, which is exactly the frozen dictionary the serializer needs. The amortized cost of an operation is $O(\log T)$ for T terms; the worst case is $O(T)$ for a single operation.

2.6.2 Hash Table and `std::map`

The hash dictionary provides expected $O(1)$ lookup and no ordered traversal; it is the speed baseline. The `std::map` dictionary is a red-black tree with worst-case $O(\log T)$ lookup and stable behavior; it is the balanced-tree control. Both implement the same three-operation interface, and all three produce byte-identical index files. The comparison of their build costs is reported in Section 4.2.

2.6.3 Comparison

The splay tree pays for its ordered traversal with rotations and a higher comparison count; the hash table is faster but cannot produce a sorted term list without an extra sort; `std::map` sits between them with predictable costs. None of these differences changes the persisted index, because the dictionary is frozen before serialization.

2.7 Disk-Based B+ Tree

For a vocabulary that does not fit in memory, the query side can use a disk B+ tree *sidecar* that maps each term to its posting offset and block length. The tree is a 4 KiB page file: page 0 is the header, and each later page is a leaf or an inner node. Leaves are linked by `nextLeaf` in key order. Keys are variable-length strings; the fanout is determined by the byte budget.

The value of a leaf entry is a 64-bit integer that packs the posting block offset and the block length:

$$\text{value} = (\text{offset} \ll 21) \mid \text{blockBytes}. \tag{5}$$

The length field is 21 bits, which is ample for the block sizes in this project.

The tree maintains five invariants, checked by `validate()`. The routing interval is left-closed, $[\text{key}_{i-1}, \text{key}_i)$, because a separator key is a real copy of the smallest key in the right subtree. With variable-length keys, a count-based split does not guarantee a 50% byte fill; the implementation splits at the byte midpoint of a legal window and uses the provable lower bound `pageSize/2 - maxEntry` for the fill check. A lookup reads the root-to-leaf path and then seeks to the posting block:

```
page = readPage(root)
while page is an inner node:
    i = upper_bound(page.keys, key)   # left-closed intervals
    page = readPage(page.child[i])
binary search page.keys for key
if found: return page.value
else: return not_found
```

A lookup costs $O(h)$ page reads and $O(h \log F)$ comparisons for height h and fanout F; it does not load the full term table. The sidecar is built by bulk-loading the sorted terms and is rebuilt whenever the index is rebuilt.

2.8 Block-Based External Construction

The in-memory builder keeps the whole dictionary and all postings in memory. For a large corpus this is the wrong bound, so the project also implements a SPIMI-style block builder. Documents are streamed one at a time; the buffer is spilled to a sorted temporary *run* when it reaches a term budget (`--block-terms`) or an estimated byte budget (`--block-bytes`). Runs are then merged in passes with at most `--merge-fan-in` inputs per pass. The term metadata and postings are streamed to temporary files and assembled into the final index, so the process does not rebuild the full index or all compressed blocks in memory.

```
for each document D in manifest order:
    add tokens of D to buffer
    if buffer.terms >= term_budget or buffer.bytes >= byte_budget:
        sort buffer; write run; clear buffer
```

```
if buffer is non-empty: sort buffer; write run
merge runs in passes of at most fan_in runs
```

The memory bound is the block budget plus document metadata, run paths, file buffers, and one document. The budget is checked at document boundaries, so a single document can exceed it. Because every run boundary falls between documents, document IDs increase across runs, and the postings of the same term can be concatenated during the merge without reordering.

On the Shakespeare corpus with a budget of 4,096, the builder produces 17 runs, peaks at 5,129 live terms, writes 4,166,556 bytes of temporary run data, and produces an index byte-identical to the in-memory build. The merge cost is $O(T \cdot R)$ with a linear scan of all run cursors, or $O(T \log R)$ with a heap; the implementation uses the simpler scan and reports the cost in Section 3.6.4.

2.9 Query Processing and Threshold Design

2.9.1 Normalization Pipeline

The query text is tokenized, filtered, and stemmed by the same functions used at build time. The index header flags decide whether stop-word filtering and stemming are applied. Every term that is filtered, missing from the dictionary, or dropped by a threshold is reported in an explanatory message, so an empty result is not silent.

2.9.2 Word, Phrase, and Boolean Queries

A word query must normalize to exactly one term. A multi-word query uses `--words`; a phrase query uses `--phrase`. For a phrase, the engine starts from the term with the smallest document frequency, takes its documents as candidates, and verifies that the remaining terms occur at `start+1`, `start+2`, and so on. Positions are sorted, so each check is a binary search.

```
phrase_query(terms):
    choose t0 with the smallest df
    for each document d in postings(t0):
        for each start position p of t0 in d:
            if for every i in 1..k-1: p+i is in positions(t_i, d):
                emit d; break
```

A Boolean query requires all searchable terms by default. The score vector is `(matched, tfSum, docID)`, sorted by descending matched count, descending term-frequency sum, and ascending document ID, which makes the result order deterministic. `--min-match k` requires at least k matching terms; if k exceeds the number of searchable terms, it is clamped to that number.

2.9.3 Thresholds

All four thresholds act at query time; the index is built once. Their positions and defaults are listed in Table 1.

Table 1: Query thresholds: where they act and their defaults.

Option	Default	Stage	Meaning
`--max-df`	1.0	query-term filter	drop a term whose df ratio exceeds the threshold
`--min-df`	1	query-term filter	drop a term whose df is below the threshold
`--min-match`	0	Boolean condition	require at least k matching terms; 0 means all
`--top-x`	0	result truncation	keep only the first x documents after ranking

Phrase queries deliberately bypass `--max-df` and `--min-df`: a phrase must match as a whole, and dropping any term changes its meaning. A query whose terms are all stop words always

returns nothing, because stop words never enter the index; the program distinguishes "stop word" from "not in the dictionary" in its explanation.

2.10 Summary of Main Data Structures

Table 2 summarizes the structures that carry the invariants and costs discussed above. The table is a specification reference; measured numbers appear in Chapter 3.

Table 2: Main data structures, invariants, and operation costs.

Structure	Key fields / layout	Invariants	Operations and cost
Posting entry	`docID` u32, `posStart` u32, `tf` u32	document IDs strictly increasing; `tf` equals the number of positions	append $O(1)$; phrase check $O(\log \mathrm{tf})$ per position
Document table	`docID`, `keptTokens`, `title`	IDs dense from 1 to N; `keptTokens` sums to the total positions	lookup by ID $O(1)$
Splay dictionary	term, posting block, left/right child	binary-search-tree order; splay moves the accessed node to the root	`upsert`/`find` $O(\log T)$ amortized, $O(T)$ worst; `sortedEntries` $O(T)$
Hash dictionary	term, posting block, bucket array	load factor bounded; rehash preserves entries	expected $O(1)$ lookup; no ordered traversal
B+ tree node	leaf flag, `keyCount`, `nextLeaf`, keys, values/children	left-closed routing intervals; byte fill lower bound; leaf chain in key order	lookup $O(h)$ page reads and $O(h \log F)$ comparisons
Index header	`magic`, `version`, `flags`, `docCount`, `termCount`, `tokenCount`	flags replay the build-time normalization; counts match the tables	read once $O(1)$
Block run	sorted sequence of (term, partial postings)	runs are disjoint by document range; document IDs increase across runs	merge $O(T \log R)$ with a heap, $O(TR)$ with a linear scan

3 Testing Results

3.1 Test Environment and Method

All measurements were taken on macOS 27.0.1 (build 26A434), an Apple M3 with 16 GiB of memory, CMake 4.4.3 (Unix Makefiles), and Apple clang 21.0.0. The code is C++17, compiled with `-Wall -Wextra` in Release mode (`-O3 -DNDEBUG`). Catch2 v3 is linked only into the tests; the `build/mse` binary (329,136 bytes) links only system libraries.

CTest runs 18 tests: 8 Catch2 unit-test executables (46 `TEST\_CASE`s and 85,155 assertions), 2 corpus smoke tests, and 8 independent cross-check tests. All 18 pass in 40.90 seconds. The cross-checks are not a second run of the same code: `tools/check\_wordcount.py` re-tokenizes and re-stems the corpus to produce the expected CSV, `tools/brute\_force.py` scans the text layer and independently parses the binary index, and `tools/check\_phrases.py` brute-forces sampled phrases on the Python token stream. The golden sample is the four-document example from the course notes, checked posting by posting.

The test-case table below is a representative subset. It records the purpose, expected result, actual behavior, root cause for corrected defects, and current status of each case. The status vocabulary is deliberately small: *Pass*, *Corrected*, and *Pending*.

3.2 Test Cases

Table 3: Representative test cases, expected and actual behavior, root causes of corrected defects, and current status.

ID	Purpose	Expected result	Actual behavior	Possible cause	Status
TC-01	Tokenizer boundary unit tests: apostrophes, hyphen, digits, case, and options (20 cases).	Spec tokens: `'Tis` → `tis`, `friends'` → `friends`, `don't` unchanged, `to-day` → `to` and `day`, `1660s` → `s`; all lowercase.	All 20 cases pass.	—	Pass
TC-02	Text-layer and token-stream differential: C++ versus an independent Python implementation over 195 documents.	195 text files byte-identical; 866,626 tokens with identical per-document counts.	The Python oracle initially reported 4,340 fewer tokens than C++; after the fix, the two streams are identical.	The oracle's line-marker predicate tested only the first character, so lines such as `[To HELENA] The best wishes ...` were mistaken for headings. It now requires the first and last characters to match.	Corrected
TC-03	HTML preprocessing edge cases: malformed `</TITLE`, table-wrapped poem, and bracket stage directions.	Full text extracted; 610 bracket stage directions relocated (57 whole-line and 553 line-start), 8 mid-line forms retained; net change −849 tokens.	The first extractor returned zero lines for `elegy.html`; bracket fragments polluted body text. After the fixes, the text layer matches and the default count is 866,626.	Malformed `</TITLE` kept the parser in the head; line-start `[...]` was not relocated; table row numbers were kept. Treat `<body>` as the end of the head, relocate line-start brackets, and drop numeric row cells.	Corrected

ID	Purpose	Expected result	Actual behavior	Possible cause	Status
TC-04	Word-count CSV differential in the default and filtered configurations.	Byte-identical CSVs; filtered: 556,866 kept and 309,760 removed; `tf\_share` uses the kept-token total.	The default CSV matched; the filtered CSV initially differed on every row; after the fix both match.	`tokens` counted pre-filter tokens, so `tf\_share` used the wrong denominator. Count only kept tokens and accumulate `stopwordTokens` separately.	Corrected
TC-05	Stop-word module and automatic derivation: 39-word list, boundary words, missing file, and `derive(0.40, 0.001,0.93)`.	List accepted; boundary words preserved; missing file errors; derivation yields 39 words with an overlap of 35.	All pass.	—	Pass
TC-06	Stemmer unit tests and full-vocabulary differential against NLTK's MARTIN extension mode (25,705 forms).	Paper examples, contractions, and guardrails (about 100 cases) pass; all 25,705 forms byte-identical.	Initially 21 expectations were wrong and 41 forms were over-stemmed (`agreement` → `agreem`); after the fixes all pass.	The paper's example column is a per-rule local effect, not the final output; suffix matching must stop at the first matching suffix and stop when its condition fails.	Corrected
TC-07	Splay dictionary operations: 20,000 mixed operations against a hash oracle over 300 terms.	Identical lookup results and no crash.	Initially a segmentation fault during insert; after the fix, identical results.	The insert branch kept a `Node*&` reference to a child slot; splay rotations rewired the slot, so the reference dangled. Copy the new node pointer to a local before splaying.	Corrected
TC-08	Golden sample, serialization round-trip, and three-dictionary determinism.	Golden4: 4 documents, 25 tokens, 11 terms, 24 postings, 750 bytes; round-trip identical; splay, hash, and `std::map` byte-identical.	Loading the golden index initially failed with "index file is truncated"; after the fix, the round-trip passes and all three dictionaries produce identical files.	`Reader reader( readWholeFile (path))` bound a reference to a temporary vector; store the file contents in a named variable first.	Corrected
TC-09	Full brute-force index cross-check: independent Python scan and binary parser, formal and raw indexes.	14,937 formal terms and 25,705 raw terms; df, tf, docIDs, and positions all match.	OK.	—	Pass
TC-10	Phrase queries: 21 query unit tests plus 305 sampled and 303 checked random phrases.	0 mismatches; specified stop-word phrase behavior.	0 mismatches.	—	Pass

ID	Purpose	Expected result	Actual behavior	Possible cause	Status
TC-11	Query threshold matrix: 15 queries, 201 measurements, four thresholds.	`love --max-df 0.7` → 0; `sweet sword moon --min-df 50` → 90; `king queen` min-match 1/2/3 → 43/33/33; `love` top-x → 5/10/20; `the` → 0.	Matches.	—	Pass
TC-12	Disk B+ tree sidecar: `dump --verify-all` over 14,937 terms.	All 14,937 terms resolve through the B+ tree to the same postings as a full load; 0 mismatches.	Initially the I2 interval check failed, the 50% fill invariant was violated for skewed keys, and the lazy header printed "1 terms"; after the fixes, 0 mismatches (114 page reads and 29,819 cache hits).	Separator keys are copies (left-closed intervals); count-based splitting does not guarantee byte fill (use the byte midpoint); the lazy path must take `keyCount()` from the sidecar.	Corrected
TC-13	v2 compression and independent parse.	An independent parser reads v2; 14,937 terms match v1; varint boundary tests pass; the file is smaller.	Initially all posting offsets were shifted; after the fix, all terms match and the formal index is 1,725,694 bytes (46.82%).	The offset computation omitted the term table's own size; the v1 helper already computes it.	Corrected
TC-14	RFC second corpus: 3 documents, the same binary, and zero code changes.	Stats, build, query, and cross-check pass; 58 derived stop words; `must`, `may`, and `should` excluded.	Matches; the derived table has 58 words (an earlier draft's 56 was a documentation error, corrected here).	—	Pass
TC-15	500,000-document scale: block build versus in-memory build.	Both produce 427,603,247-byte v1 indexes with identical SHA-256; block peak 1.01 GiB versus 2.00 GiB.	Matches.	—	Pass
TC-16	400,000,000-distinct-word end-to-end scale.	Complete build and query for 400,000,000 distinct words within resources.	Not run end-to-end. The B+ tree is measured to 50,000,000 keys and extrapolated to 400,000,000 (height 4, four page reads); block build is measured to 4.9 million terms.	Resource cost; missing direct SPIMI-to-B+ tree integration and distributed sharding.	Pending

Note: *Corrected* means a defect was found and fixed; the actual-behavior column records the pre-fix observation, and the expected column records the post-fix result. *Pass* means the final run passed without a code change. The table is a representative subset of the 18 CTest tests and the 201 threshold measurements.

3.3 Word Frequency and Stop-Word Results

3.3.1 Word-Frequency Results

The default configuration counts body lines only. The 195 documents produce 866,626 tokens, of which 25,705 are distinct and 9,753 occur exactly once. Table 4 gives the core counts. The ten most frequent terms are listed in Table 5; the Zipf plot in Figure 3 shows the

full distribution.

Table 4: Word-frequency statistics in the default configuration.

Metric	Value
Documents	195
Body tokens	866 626
Distinct terms	25 705
Terms occurring once (hapax)	9 753

Table 5: The ten most frequent terms in the default configuration, with document frequency and two ratios.

Rank	Term	tf	df	df ratio	tf share
1	`the`	28 436	177	0.9077	0.032812
2	`and`	25 553	189	0.9692	0.029486
3	`i`	21 131	154	0.7897	0.024383
4	`to`	20 302	187	0.9590	0.023426
5	`of`	16 976	175	0.8974	0.019589
6	`a`	14 527	138	0.7077	0.016763
7	`you`	13 998	75	0.3846	0.016152
8	`my`	12 934	159	0.8154	0.014925
9	`that`	11 513	172	0.8821	0.013285
10	`in`	11 479	181	0.9282	0.013246

Note: df ratio is df/195 and tf share is tf/866,626; values are taken from the authoritative copy of `wordcount-default.csv`.

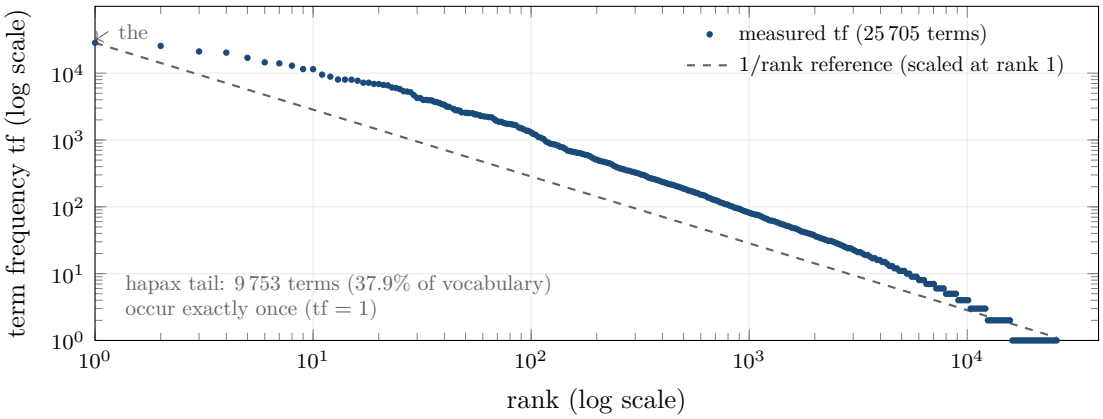


Figure 3: Word frequency versus rank on the Shakespeare corpus (log-log). The measured points approximate a straight line, consistent with Zipf's law; of 25 705 terms, 9 753 occur exactly once (37.9% of the vocabulary), forming the long tail that the inverted index must cover.

The top ten terms are almost all function words and together cover 20.41% of the tokens. The log-log curve in Figure 3 shows a steep head and a long low-frequency tail. Terms occurring once make up 37.94% of the vocabulary but cover only 1.125% of the tokens. Frequency alone therefore cannot define a stop word: the next step compares document coverage and the distribution across documents.

3.3.2 Stop-Word Candidate Sensitivity

Table 6 scans the document-frequency-ratio threshold from 0.90 down to 0.40 while holding the term-frequency share at or above 0.001.

At 0.75 the band contains only nine pure function words; at 0.65 the content word `love` enters (df ratio 0.651); at 0.40 words such as `time` and `thou/thee` would be removed even though they carry meaning. A purely statistical rule can reliably frame the "most stop-word-like" band

Table 6: Sensitivity of the stop-word candidate band to the document-frequency-ratio threshold (tf share fixed at ≥ 0.001).

df-ratio threshold T	Candidates	Added over previous	New terms
0.90	4	—	`the and to in`
0.75	9	+5	`i of my that but`
0.70	14	+5	`a is not for with`
0.65	16	+2	`so love`
0.60	21	+5	`me be thou thy thee`
0.50	31	+10	`it this as by all do on from which when`
0.40	48	+17	`his have what no if are o or more then their hath than yet mine time doth`

Note: the rule is df ratio $\geq T$ and tf share ≥ 0.001; the static 39-word list is the manual selection made from the 48-word band at the 0.40 level.

but cannot decide where the line should be. The static list is the manual choice on the 48-word band; the next subsection replaces that manual step with the rate entropy.

3.3.3 Automatically Derived Stop Words

Adding the "flat" feature tests whether the rule can reproduce the manual list automatically. Table 7 compares four threshold settings with the static 39-word list.

Table 7: Threshold sweep of the three-feature derivation: differences from the static 39-word list.

(df ratio, rate entropy)	Derived	Overlap	Static only	Derived only
(0.75, 0.93)	9	9	30	—
(0.50, 0.95)	23	23	16	—
(0.40, 0.93)	39	35	`more or than their`	`hath o thee thou`
(0.30, 0.93)	50	35	`more or than their`	`at hath he him how let now o thee there thou was well would you`

Note: tf share is fixed at ≥ 0.001, and the first column gives the df ratio. The (0.75,0.93) setting reproduces the nine-word two-feature band; the (0.40,0.93) setting reproduces the 39-word band that contains the static list.

The (0.40,0.93) setting derives 39 words and overlaps the static list in 35. The differences are four explainable pairs: the automatic list adds the archaic function words `hath`, `o`, `thee`, and `thou`, while the static list adds `more`, `or`, `than`, and `their`, which also serve as comparison and conjunction words. The former are wide and flat in the source text; the latter are a semantic judgement by the curators. The three-feature rule turns "is this a stop word?" from a manual decision into a recomputable test, and the result differs from the manual table in only four words, each of which can be explained. Changing the threshold changes only the strictness (9/23/39/50 words), not the structure of the rule. All four output files carry a provenance header and no timestamp, so a parameter change can be rerun and compared directly.

Figure 4 plots the rate entropy of function words, content words, and proper nouns and marks the 0.93 threshold.

The function words `the`, `and`, `i`, `to`, `of`, `is`, `it`, and `that` all exceed 0.95 (`the` 0.9748, `and` 0.9789); the content words `love`, `king`, `heart`, and `sweet` fall between 0.82 and 0.93 (0.9271, 0.8260, 0.8313, 0.8925); the proper nouns `osric` and `banquo`, which occur in one document each, are zero by definition. The 0.93 line separates function words from content words. The word `man` has rate entropy 0.9388, slightly above the line, but its df ratio is only 0.2359, so the "wide" condition keeps it out of the band. The three conditions are conjunctive, and rate entropy supplies the "is the distribution flat?" dimension that the other two features lack.

The final stop-word list is:

```
the and i to of a my that in is not me it for with be his this but have as so what
by no all do if are on from or which more then their when than yet
```

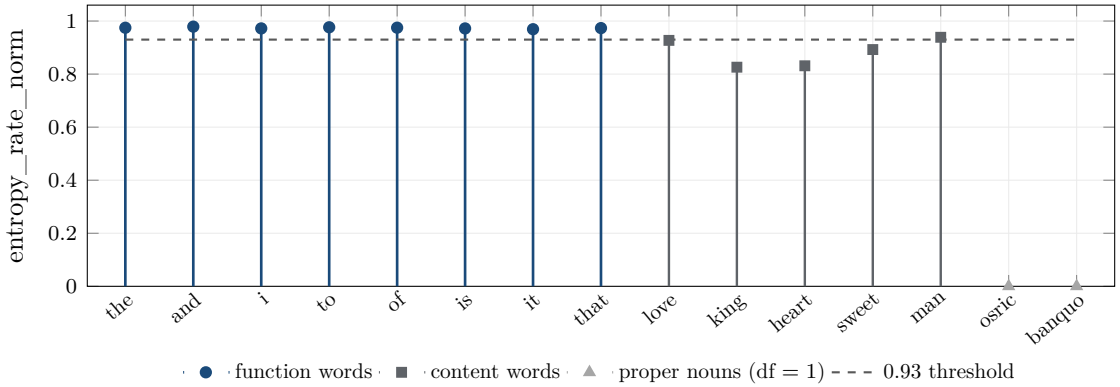


Figure 4: Rate entropy as a stop-word test: function words cluster near 1 (all above 0.95), proper nouns with df = 1 are zero by definition, and content words fall in the lower band 0.83–0.94. A single 0.93 line separates function words from the rest; the few content words above the line are still blocked by the df-ratio gate (≥ 0.4), so the feature can replace manual review.

3.4 Inverted Index and Query Results

3.4.1 Index Construction Results

The formal index uses the 39-word stop-word list and applies stemming. Its final build statistics are given in Table 8.

Table 8: Final build statistics for the formal inverted index.

Metric	Result
Documents	195
Kept tokens	556 866
Terms	14 937
Posting entries	117 704
Positions	556 866
v1 index size	3 685 655 bytes
v2 index size	1 725 694 bytes
Build time	0.19 s

Splay, hash, and `std::map` dictionaries were each used to build the same formal index; the three files are byte-identical. The comparison of their build costs is deferred to Section 4.2, because it is an analysis of alternatives rather than a result of the required build. Format v2 reduces the formal index to 1,725,694 bytes, about 46.82% of v1.

3.4.2 Query Results

The formal index is used to test word, phrase, and Boolean queries and to check whether an empty result is explained. Representative queries are listed in Table 9.

Table 9: Representative query results on the formal index.

Query	Type	Documents	Result
`osric`	word	1	document 30, *Hamlet*; the source line can be located
`love`	word	138	high-frequency content word; all containing documents
`good night`	phrase	23	the two terms must be adjacent
`king richard`	phrase	7	both terms exist and pass the position check
`my lord`	phrase	42	`my` is filtered as a stop word; only `lord` is queried
`to be or not to be`	phrase	0	all terms are stop words
`king queen`	Boolean	33	both terms are required by default
`the`	word	0	stop words never enter the index
`notaword`	word	0	the term is not in the dictionary

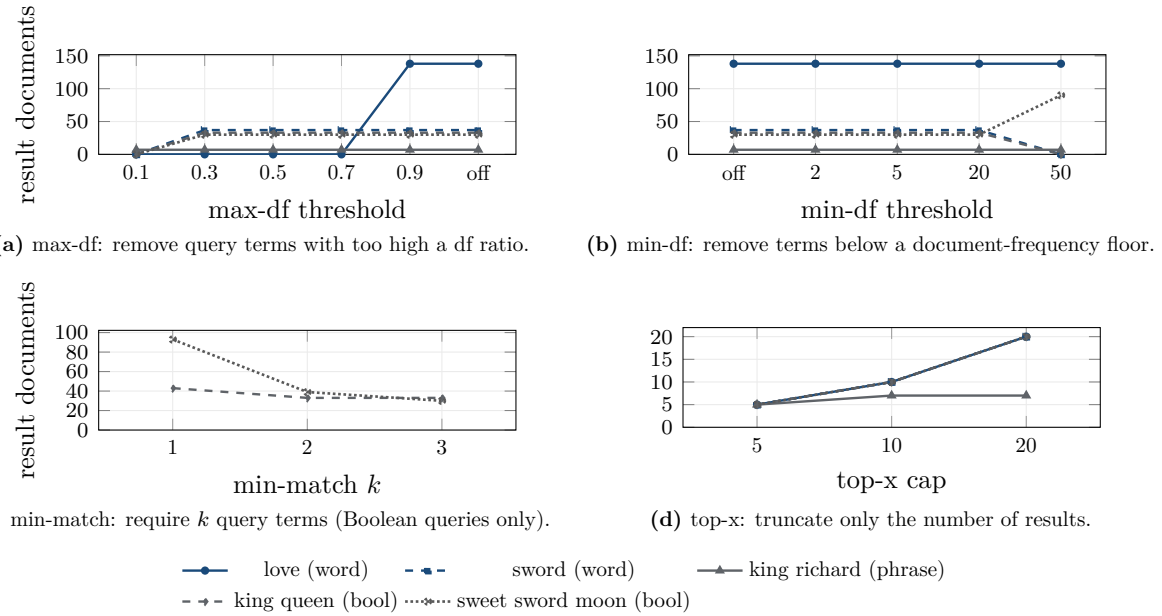
A word query returns the document IDs and titles that contain the term. Results are ordered deterministically by matched-term count, total term frequency, and document ID. With `--pretty`, the program reruns the text pipeline and prints the source line containing the match.

A phrase query verifies that the terms occur consecutively in the stop-word-filtered token stream; it is not enough to intersect the posting lists. `good night` returns 23 documents, and only documents in which the two terms are adjacent are kept. Because positions are defined on the filtered stream, `my` in `my lord` occupies no position, and the system effectively executes a word query for `lord`. A phrase made only of stop words has no locatable term and returns nothing.

A Boolean query requires all query terms by default, so `king queen` returns 33 documents. With `--min-match 1`, either term is enough and the result rises to 43. When a query is empty, the program distinguishes "stop word not in the index" from "term not in the dictionary" instead of collapsing both cases into one message.

3.5 Threshold Experiment Results

All four parameters act only at query time. With every parameter off, the same query gives the baseline. The experiment changes one parameter at a time over 15 queries that cover high-, medium-, and low-frequency words, proper nouns, ordinary phrases, phrases containing stop words, Boolean queries, and stop words. Every query is tested with five `max-df` levels, four `min-df` levels, and three `top-x` levels; the minimum-match count applies only to the Boolean queries `king queen` and `sweet sword moon`, with three levels. This gives $15 + 15 \times (5 + 4 + 3) + 2 \times 3 = 201$ measurements. Each record stores the result count, the Jaccard similarity to the baseline, the number of explanation messages, and the elapsed time, all on the same formal index. Figure 5 shows the effect of each parameter on the number of returned documents.



(a) max-df: remove query terms with too high a df ratio.

(b) min-df: remove terms below a document-frequency floor.

(c) min-match: require k query terms (Boolean queries only).

(d) top-x: truncate only the number of results.

Figure 5: The four thresholds change query semantics rather than simply the number of results: max-df/min-df remove query terms (a single-term query can drop to zero at the critical level, and a multi-term query can return more after a term is removed), min-match changes the Boolean condition, and top-x only truncates the output; phrase queries are insensitive to the first two parameters.

Table 10 merges the representative readings by parameter.

The maximum and minimum document-frequency thresholds filter query terms, not result documents. A word query has only one term, so removing that term makes the result zero. `love` has a df ratio of 0.7077, so at the 0.7 level the result drops from 138 to 0; `sweet` has a

Table 10: Effect of the four query thresholds on the returned results.

Option	Representative query	Value: baseline → result	Jaccard
`--max-df`	`love`	0.9/0.7: 138 → 138/0	1.00/0.00
`--min-df`	`sweet sword moon`	50: 30 → 90	0.3333
`--min-match`	`king queen`	1/2/3: 33 → 43/33/33	0.7674/1.00/1.00
	`sweet sword moon`	1/2/3: 30 → 93/39/30	0.3226/0.7692/1.00
`--top-x`	`love`	5/10/20: 138 → 5/10/20	0.0362/0.0725/0.1449

Note: Jaccard is the intersection-over-union of the result set with the baseline set; two empty sets are treated as equal.

ratio of 0.4615 and also drops to zero at the 0.3 level.

A multi-term query can lose a constraint before it loses results. `sweet sword moon` has a baseline of 30 documents. With `--min-df 50`, `sword` (df 37) and `moon` (df 35) are removed, leaving `sweet`; the three-term conjunction degenerates into a word query and the result rises to 90. This is not an increase in precision: the term filter has changed the meaning of the query.

The minimum-match count controls the Boolean condition on the result side. $k = 1$ is a union, $k = 2$ requires at least two terms, and $k = 3$ requires all three; when k exceeds the number of searchable terms, the program clamps it to that number. Result truncation is simpler: after ranking, only the first x documents are kept, so `love` returns 5, 10, and 20 documents without changing their relative order.

Phrases and stop words provide the two boundary cases. A phrase query bypasses the maximum and minimum df thresholds, so `king richard` stays at 7 documents at every level; removing either term would destroy the phrase. Stop words never enter the index, so `the` and `of` return zero at every threshold. Of the 201 measurements, 23 have Jaccard zero; each occurs after all effective query terms have been removed, and the explanation records the corresponding terms and the reason. No empty result is unexplained.

Finding: The four thresholds are not the same kind of "tightening." The maximum and minimum df thresholds change which terms participate, `min-match` changes the Boolean condition, and `top-x` only changes the output range. Removing terms can make a result empty or, by weakening a conjunction, larger.

3.6 Bonus: Scalability Experiments

The bonus question has two independent scale axes: 500,000 files stress the build side and the document table, while 400,000,000 distinct words stress the dictionary and the posting lists. The project implements three mechanisms for these axes—SPIMI-style block construction, a disk-based B+ tree dictionary, and the delta-coded v2 index format—and runs the scale experiments that fit on one machine.

3.6.1 Scale Boundaries and Measured Range

The 500,000-document synthetic corpus contains 4,908,747 distinct terms. Its full in-memory index peaks at 2.00 GiB. Scaling that figure linearly by term count to 400,000,000 distinct words would require about 160 GiB, which a single machine of this class cannot provide. An in-memory build therefore cannot answer the second axis directly.

The file axis is measured end to end: statistics, block construction, and the in-memory build all run on the same 16 GiB machine, and no address-space or document-table problem appears at 500,000 files. The distinct-word axis is not run end to end. The query side is measured up to 50,000,000 B+ tree keys and extrapolated from the measured trend; the build side is measured to 4.9 million terms and bounded by the O(budget) memory argument. Measured points and extrapolated points are labelled as such throughout this section.

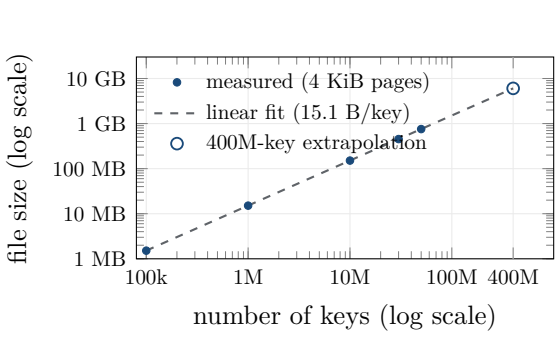
3.6.2 Disk B+ Tree Dictionary

The query-side problem is not how to search faster but how to avoid loading the complete term table. The B+ tree sidecar maps each term to a posting offset and block length, so a lookup touches only a few pages on the root-to- leaf path. The experiment measures 100,000 to 50,000,000 keys; results are in Table 11.

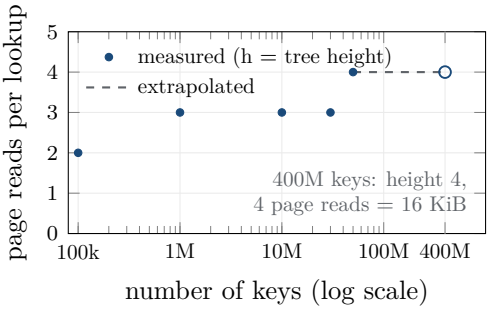
Table 11: Measured scaling of the B+ tree dictionary (4 KiB pages, bulk load, 1,000 random lookups all verified).

Keys	Height	Pages	File bytes	Bytes/key	Page reads/query
100 000	2	369	1 515 520	15.1552	2.00
1 000 000	3	3 688	15 110 144	15.1101	3.00
10 000 000	3	36 865	151 003 136	15.1003	3.00
30 000 000	3	110 593	452 993 024	15.0998	3.00
50 000 000	4	184 322	754 987 008	15.0997	4.00

Note: page reads are the actual number of pages read with a one-page cache; they equal the tree height.



(a) File bytes = 15.1 B/key × keys; strictly linear.



(b) Each lookup reads tree-height pages: even 400M keys need 4 pages = 16 KiB.

Figure 6: Scaling experiment for the B+ tree dictionary: file size is strictly linear (15.1 B/key), while page reads per random lookup equal the tree height and grow logarithmically with height. Extrapolating from the measured fanout, 400M distinct words need 4 levels, 4 pages per lookup (16 KiB), and a file of about 6.04 GB.

Across the five measured points, bytes per key stays between 15.10 and 15.16, so file size is almost exactly linear in the key count. The fanout is about 330–370 and a leaf holds about 272 keys on average, so four levels already cover 10^8-scale keys. Extrapolating the measured trend to 400,000,000 keys gives a file of about 6.04 GB, still height 4, and four page reads per lookup, about 16 KiB. Height and page reads are already measured at 50,000,000 keys; only the file size and page count are extrapolated.

3.6.3 Block-Based Construction with 500,000 Documents

On the formal Shakespeare corpus, block construction with a 4,096-term budget produces 17 temporary runs, a peak of 5,129 live terms, and 4,166,556 bytes of temporary data; its output is byte-identical to the in-memory build.

The 500,000-document experiment scales this path to realistic small-file size. The synthetic corpus has 500,000 files, 20,000,000 tokens, and 4,908,747 distinct terms. Block construction produces 4,855 runs; scanning takes 5.47 s, merging 125.08 s, for a total wall-clock time of 168.16 s and a peak memory of 1.01 GiB. The in-memory build takes 76.75 s and peaks at 2.00 GiB. Both produce a v1 index of 427,603,247 bytes with the same SHA-256.

Block construction almost halves peak memory, at the cost of a longer merge. The current merge finds the smallest term by scanning all 4,855 cursors, about $O(T \cdot R)$; a heap-based k-way merge would reduce this to $O(T \log R)$. The merge time and the temporary space are the engineering price of the memory bound.

Table 12: Build results for 500,000 documents and 20,000,000 tokens.

Build	Index bytes	Build phase	Peak memory	Output
Block, v1	427 603 247	scan 5.47 + merge 125.08 s	1.01 GiB	SHA-256 identical
Block, v2	284 560 009	scan 5.42 + merge 123.31 s	1.46 GiB	delta-coded
In-memory, v1	427 603 247	build 44.93 s, total 76.75 s	2.00 GiB	baseline

Note: the block-built and in-memory v1 indexes are identical under `cmp` and SHA-256.

3.6.4 Compression Results of the v2 Index Format

The v2 format changes only the posting region: document IDs and positions are delta-coded, and term frequencies and deltas use byte-aligned variable-length codes. On the main corpus the index drops from 3,685,655 to 1,725,694 bytes, 46.82% of v1, while build time stays at 0.19 s and query output is unchanged.

On the 500,000-document corpus the posting region is only 56.1% of the total file, so the term table becomes the main contributor to size and the overall ratio is worse: 427,603,247 bytes fall to 284,560,009 bytes, 66.55% of v1. The posting region itself compresses to about 59.6%, close to the 61.8% on the main corpus.

Table 13: v2 compression results and the main cost.

Corpus	v1 bytes	v2 bytes	v2/v1	Saving	Cost
Shakespeare	3 685 655	1 725 694	46.82%	53.18%	0.19 s build
500k synthetic	427 603 247	284 560 009	66.55%	33.45%	1.46 GiB peak (v1: 1.01)

Note: the 500k ratio is dominated by the term table, which v2 does not compress.

Compression suits storage-constrained settings where CPU time and build memory are relatively abundant. The 3.7 MB main index fits in memory entirely, so the v2 benefit is mainly on disk and in transfer; on the 500k corpus the compressed build raises peak memory from 1.01 to 1.46 GiB, so compression does not win on every resource dimension. These trade-offs are analyzed further in Section 4.5.

3.7 Summary of Testing Results

The required tasks pass their correctness checks: the tokenizer, stemmer, dictionary, index, and query tests are green, and independent Python implementations agree with the C++ output on the word-count CSV, the full posting tables, and 303 phrase queries. The four thresholds behave as specified, including the counterintuitive case in which filtering query terms increases the number of results. The bonus experiments show that the 500,000 file axis is measured end to end, while the 400,000,000 distinct-word axis is covered by measured B+ tree and block-build components plus a clearly labelled extrapolation. The defects found during testing are listed in Table 3; each one was fixed and rerun, not merely described.

4 Analysis and Comments

The results show that the implementation completes the course tasks and that two scale paths are available. This chapter asks two further questions: what do the measured costs mean asymptotically, and how far is the current engine from a more general search system?

4.1 Time and Space Complexity

Table 14 gives upper bounds for the main operations. N, T, P, D, and H denote the number of tokens, terms, postings, documents, and matching documents; k is the number of query terms, R the number of temporary runs, B the block budget, and C the cost of one dictionary comparison. In phrase verification, $\mathrm{tf}_{\max}$ is the largest frequency of a candidate term within one document.

Table 14: Time and space upper bounds for the main operations.

Operation	Time	Space
Index build (in memory)	$O(N + \sum \mathrm{df} \cdot C)$	$O(T + P + N)$
Index build (block)	$O(N + RB \log B + TR)$	$O(B +$ postings of the current run$)$
Word query	$O(\log T + \mathrm{df} + H \log H)$	$O(D)$
Phrase query	$O(\mathrm{df}_{\min} \mathrm{tf}_{\max} k \log \mathrm{tf}_{\max} + H \log H)$	$O(D + k)$
Boolean query	$O(\sum \mathrm{df} + H \log H)$	$O(D)$
v2 encoding	$O(P + N)$	$O($one block$)$
B+ tree lookup	$O(h \log F)$, plus $O(h)$ page reads	$O(h)$ page cache

Note: F is the B+ tree fanout and h its height.

In-memory index construction spends most of its time on one dictionary access per token, and its output keeps the term table, the postings, and the positions at the same time. The three dictionaries produce indexes of the same size; only the comparison count and the comparison constant change. Block construction uses B to bound the live terms at any moment, but it does not remove the cost: a smaller B means more runs, and sorting and merging become visible. The disk B+ tree makes a different trade: a query reads the $O(h)$ pages on the root-to-leaf path and no longer requires the full term table in memory.

On the query side, the main cost is scanning the relevant postings and sorting the matching documents. A phrase query starts from the term with the smallest document frequency, which shrinks the candidate set, but it still verifies k positions in every candidate. The thresholds relate to complexity directly: `max-df` and `min-df` remove terms before the posting scan and can reduce later work, whereas `min-match` and `top-x` act after scoring; `top-x` only reduces the output and does not save the scan or the sort. In this implementation the thresholds are first a tool for changing query semantics, not a uniform performance switch.

4.2 Comparison with Alternative Data Structures and Algorithms

Table 15 compares the three dictionary implementations on the same formal index.

Table 15: Build cost of three dictionary implementations (formal configuration, three builds each).

Dictionary	Comparisons	Rotations	Rehashes	Build time (in-process / external median)
Splay tree (main)	6 392 724	5 850 795	0	0.19 s / 0.2610 s
Hash table	688 494	0	11	0.09 s / **0.1603 s**
`std::map` (control)	9 435 694	0	0	0.14 s / 0.2084 s

Note: comparison, rotation, and rehash counts are in-process counters; external wall-clock time includes process start and file I/O, so it is comparable only in order of magnitude.

The hash table is fastest, `std::map` is second, and the splay tree is slowest. The splay tree pays for 5.85 million rotations, but its in-order traversal directly yields a sorted term table.

The project therefore uses the splay tree as the main dictionary, the hash table as the speed baseline, and `std::map` as the balanced-tree control. The second format reduces the index to 1.73 million bytes, about 46.82% of v1.

The v1 and v2 formats represent a different trade-off. Version 1 stores fixed- width integers and is simple to parse; version 2 delta-codes document IDs and positions and uses byte-aligned variable-length codes, cutting the formal index by 53.18% with no change in query output. On the 500,000-document corpus, however, the term table dominates and the overall ratio is only 66.55%. The right choice therefore depends on whether storage or build memory is the binding constraint.

The in-memory tree and the disk B+ tree answer different questions. The splay tree is appropriate when the whole term table fits in memory and the build order is unknown; the B+ tree sidecar is appropriate when the table does not fit and a cold query must avoid loading it. The sidecar is 237,568 bytes for 14,937 terms, and a cold lookup reads 0.225% of the bytes of a full load (114 page reads versus 29,819 cache hits during the verification pass). The B+ tree is not a replacement for the in-memory dictionary; it is the boundary mechanism for the next scale.

The stemmer comparison is also informative. The implementation follows the original Porter algorithm [1] and is checked against NLTK's `MARTIN\_EXTENSIONS` variant over all 25,705 forms. The original paper's examples describe local rule effects, not necessarily the final stem; the canonical C implementation and Snowball add fixes and extensions. The 41 over-stemmed forms found during testing came from reading the paper's example column as final output. The corrected implementation matches the extended variant, which is the appropriate target for a reproducible pipeline: the choice of variant matters less than stating which one is used.

4.3 Generalization: A Second Corpus

The second corpus contains IETF RFC 1035 (DNS), 5321 (SMTP), and 7231 (HTTP semantics). A generic importer converts the three documents to plain text; the manifest and the text layer are the only interface. The same C++ binary, without any code change, performs statistics, indexing, querying, and the independent cross-check.

Table 16: End-to-end measurements on the RFC corpus.

Metric	Raw configuration (no filter, no stemming)	Formal configuration (58 stop words + stemming)
Documents	3	3
Tokens	75 250	45 370
Terms	4 328	2 660
Postings	7 006	4 594
Index bytes	510 181	308 164
`brute_force.py` cross-check	all 4 328 terms match	all 2 660 terms match

Note: the RFC corpus has 9,946 lines and 81,848 whitespace-separated words.

The RFC corpus is the clearest demonstration that there is no universal stop-word list. Applying the same three-feature rule $(0.40, 0.001, 0.93)$ derives 58 stop words for the RFCs, overlapping the 39-word Shakespeare list in only 26. The RFC 2119 keywords `must`, `may`, and `should` occur in all three documents (df ratio 1), but their rate entropies are 0.809, 0.899, and 0.885, below 0.93, so they are not mistaken for stop words. Conversely, `server`, `page`, and `rfc` have rate entropy close to 1 and are included in the derived list.

Queries behave as expected: `zone` matches all three documents, `postmaster` matches the SMTP document and normalizes to `postmast`, and in `name server` the term `server` is filtered by the 58-word list; the program explains this and queries only `name`. Conceptually, the engine and the corpus-production stage are separate: a new corpus needs new producer rules and stop-word thresholds, but not changes to the index or query code.

Generalization has limits. The RFC plain-text import does not clean page footers, so `page` from `[Page N]` enters the vocabulary and becomes a stop word. This noise belongs to the producer side, not the engine side. A new corpus still requires rechecking tokenization, document granularity, and the stop-word distribution; "the same binary runs" does not mean "no data engineering is needed."

4.4 Limitations and Possible Improvements

Table 17 lists the main boundaries of the current implementation. Phrase semantics and the ranking model have the largest effect: positions are defined on the filtered token stream, so `my lord` degenerates to a query for `lord`, and a phrase made entirely of stop words returns nothing. Ranking uses only the matched-term count and the total term frequency, without idf or length normalization, so long plays tend to occupy the top positions.

Table 17: System limitations, impact, and improvement directions.

Limitation	Impact	Improvement
Phrases match on the filtered token stream	Phrases containing stop words degenerate; all-stop-word phrases return nothing	Keep stop-word positions, or build a second phrase index without stop-word removal
ASCII-only tokenizer	Unicode corpora cannot be used directly	UTF-8 decoding and Unicode-category tokenization, followed by a full cross-check rerun
Unclean plain-text import	Boilerplate such as RFC page numbers enters the vocabulary	Clean on the producer side, keeping the step auditable and optional
Ranking uses only matched-term count and total tf	No inverse document frequency or length normalization	Implement TF-IDF/BM25 and redesign the relevance experiment
Single-threaded build and query	The 201 threshold measurements take about 4.05 s serially	Parallel posting scans; parallel block construction
Fixed document granularity (whole work)	Scene-level granularity is not part of the formal selection experiment	Parameterize granularity and compare scene-level with merged-short-poem stop-word distributions
No incremental index	Updating one document requires a full rebuild	Segment-based indexing with periodic merging
Single machine, no distribution	A real 400M-word deployment is still bounded by build time and disk	Shard by term range for build and query

The dictionary comparison should be stated honestly. Hashing is clearly faster than the splay tree for pure construction: about 0.19 s for the splay tree and 0.09 s for the hash table in-process, but the splay tree's in-order traversal directly yields a sorted term table and satisfies the course requirement for a self-implemented data structure. All three implementations write byte-identical indexes, which shows that the dictionary changes the construction process, not the persisted result.

The threshold result most easily misunderstood is that "tightening" a parameter can reduce the result set. In fact `--max-df` and `--min-df` filter query terms first: this can make a single-term query empty, or it can weaken a multi-term conjunction and increase the result. `--min-match` changes the Boolean condition, and `--top-x` is the only true result truncation. Phrase queries are unaffected by term-side thresholds, which follows from the requirement that the whole phrase match.

4.5 Bonus Analysis: Scalability and Boundaries

The bonus results separate cleanly into measured components and extrapolated ones. On the query side, the B+ tree is measured from 100,000 to 50,000,000 keys at 15.10 bytes per key and height 4; the extrapolation to 400,000,000 keys adds only file size and page count, not a new structural assumption. On the build side, block construction is measured on 500,000 documents and 4,908,747 terms, with peak memory of 1.01 GiB versus 2.00 GiB for the in-memory build. The measured peak for the full in-memory index implies about 160 GiB at 400,000,000 distinct words if scaled linearly by term count, which is why the in-memory path cannot answer the second axis.

The block build trades memory for merge time. Of the 168.16 s total, 125.08 s (74.4%) is merging, because the current merge scans all 4,855 run cursors to find the smallest term, about $O(T \cdot R)$. A heap-based k-way merge would reduce this to $O(T \log R)$ and is the first implementation improvement to make. The temporary space is the other cost: 4,166,556 bytes on the Shakespeare corpus and a much larger run set at 500,000 documents.

Compression and the disk dictionary optimize different resources. Version 2 cuts the formal index to 46.82% of v1 but raises the 500k build peak from 1.01 to 1.46 GiB; the B+ tree sidecar reduces cold-query I/O to 0.225% of a full load but adds a separate file and a page cache. None of these mechanisms is universally better; each is appropriate when its constrained resource is the one that binds.

The honest conclusion is therefore narrower than "a 400,000,000-word system runs." The key components scale: the dictionary is measured to 50,000,000 keys and structurally covers 400,000,000, and the block builder removes the assumption that the whole term table must fit in memory. A complete large-scale system still lacks direct SPIMI-to-B+ tree integration, distributed sharding, and fault recovery. Those are the next engineering steps, and they should be reported separately from the measured single-machine range.

4.6 Conclusions

The project implements the four required tasks on a 195-document Shakespeare corpus. Word-frequency statistics produce 866,626 body tokens and 25,705 distinct terms; a three-feature rule derives 39 stop words with an overlap of 35 against the manual list; the stemmed inverted index contains 14,937 terms, 117,704 postings, and 556,866 positions; word, phrase, and Boolean queries return document IDs, and the four thresholds are exercised over 201 measurements. The results are consistent with the specification in Chapter 2 and with the test cases in Chapter 3.

The most valuable outcome is not a single wall-clock number but the reproducibility chain. An independent Python implementation rebuilds the postings from the same text layer; three dictionaries write identical bytes; block construction and in-memory construction agree; the second index format is restored by an independent parser; and the disk dictionary matches a full load term by term. Each of these constraints exposed implementation errors that a "the output looks plausible" check would have missed, including the dangling `Node*&` in the splay insert, the temporary-reference bug in the index reader, and the shifted offsets in the v2 format.

The engine is not a general search system. Its tokenizer is ASCII-only, its phrase semantics inherit stop-word removal, its ranking has no idf or length normalization, and it has no incremental or distributed mode. These are stated as limitations in Table 17, not hidden behind the measured numbers. Within the single-machine, batch-built scope, however, the behavior is specified, measured, and reproducible.

The bonus work supports a component-level claim rather than an end-to-end one. The B+ tree dictionary and the block builder each address one resource limit, and both are measured well beyond the formal corpus. Integrating them directly, adding a heap merge, and sharding by term range are the concrete next steps toward the 400,000,000-word target.

References

[1] M. F. Porter. An algorithm for suffix stripping. *Program*, 14(3):130–137, 1980.

[2] C. D. Manning, P. Raghavan, and H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, 2008.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*, 4th edition. MIT Press, Cambridge, MA, 2022.

[4] MIT Shakespeare. *The Complete Works of William Shakespeare*. `http://shakespeare.mit.edu/`; GitHub mirror `https://github.com/TheMITTech/shakespeare`. Accessed 2026-10-20.

[5] M. F. Porter. *The Porter Stemming Algorithm: canonical C implementation*. `https://tartarus.org/martin/PorterStemmer/c.txt`. Accessed 2026-10-20.

[6] NLTK Project. *nltk.stem.PorterStemmer*. `https://github.com/nltk/nltk/blob/3.9.1/nltk/stem/porter.py`. Accessed 2026-10-20.

[7] Snowball Project. *The English (Porter2) stemming algorithm*. `https://snowballstem.org/algorithms/english/stemmer.html`. Accessed 2026-10-20.

A Source Code

This appendix documents how the source is organized and how densely it is commented. The complete listing is not reproduced here: the archive submitted with the report contains `src/`, `include/`, `tests/`, and `tools/` in full, and the module map below identifies what each file implements. The code is C++17 and builds with CMake.

A.1 Module Map

Table 18 lists the twelve translation units of the C++ core. Headers are grouped with the implementation they declare; line counts include both files.

Table 18: Source modules of the C++ core.

Module	Lines	Responsibility
`tokenizer.cpp`, `tokenizer.hpp`	167	ASCII token classes, apostrophe and hyphen rules, option flags
`corpus.cpp`, `corpus.hpp`	475	Manifest parsing, text layer, document metadata
`count.cpp`, `count.hpp`	338	tf/df statistics, kept and removed token counts, CSV output
`stopwords.cpp`, `stopwords.hpp`	84	Stop-word list loading, three-feature derivation, provenance header
`stemmer.cpp`, `stemmer.hpp`	424	Porter stemmer, contraction handling, guard rules
`posting.cpp`, `posting.hpp`	80	Posting entries, document and position lists
`index.cpp`, `index.hpp`	1 135	Index build, v1/v2 serialization and parsing, verification
`dict.cpp`, `dict.hpp`	552	Splay-tree, hash-table, and `std::map` dictionary adapters
`bptree.cpp`, `bptree.hpp`	1 211	Disk B+ tree, bulk load, point lookup, page cache
`blockbuild.cpp`, `blockbuild.hpp`	504	SPIMI-style run construction and merge
`query.cpp`, `query.hpp`	414	Word, phrase, and Boolean queries; thresholds; ranking
`main.cpp`	1 267	Command-line interface and command dispatch
Total	6 651	

The eight Catch2 test files total 1,708 lines, and the nine Python tools total 1,801 lines. The tools are independent of the C++ implementation: they re-tokenize the corpus, rebuild the postings, parse both index formats, and check phrase results, so agreement is evidence rather than a tautology.

A.2 Comment Coverage

Comment coverage is counted on the twelve core files (6,651 total lines, 6,112 non-blank lines). A line is *comment-only* if its first non-space characters begin a line or block comment; a line is *comment-bearing* if it contains a comment, whether or not it also contains code. Table 19 reports both conventions instead of choosing the more favorable one.

Table 19: Comment coverage of the C++ core.

Scope	Non-blank	Comment-only	Comment-bearing	Comment-only	Comment-bearing
`src/`	5 180	1 304	1 420	25.2%	27.4%
`include/`	932	414	519	44.4%	55.7%
Combined	6 112	1 718	1 941	28.1%	31.7%

Note: 223 lines contain both code and a comment; they are included in the comment-bearing column and excluded from the comment-only column. Measured against code-carrying lines alone, comment-only lines are 39.1%.

The combined core is 31.7% comment-bearing; comment-only lines are 28.1%. Headers are commented more densely than implementation files, which is expected: the headers state the invariants and ownership rules that the implementation assumes. The requirement that comments make up at least 30% of the code is met under the comment-bearing convention; the comment-only figure is reported as well so that the counting convention is visible. Comment density is not a substitute for the tests in Chapter 3: the cross-checks, not the comment ratio, are what establish correctness.

B Reproduction Commands and Key Outputs

All commands run from the project root. The readings come from the rebuild and rerun of 2026-10-20; raw outputs are archived in `reports.zip` under `reports/data/logs/`, and the evidence numbers (F/P) match the text. In the commands, `.../` denotes the same relative path under `reports/data/work/` (`tasks/` or `verify/`) in the archive; the complete path appears on the command line of each fact in the results files.

B.1 Environment, Build, and Tests

Environment: macOS 27.0.1 (build 26A434), Apple M3, 16 GiB; CMake 4.4.3 (Unix Makefiles); Apple clang 21.0.0, C++17, `-Wall -Wextra`, Release (`-O3 -DNDEBUG`); Catch2 v3 is linked only into the tests; `build/mse` is 329,136 bytes and links only system libraries.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j8
/usr/bin/time -p ctest --test-dir build --output-on-failure
# -> 100% tests passed out of 18
# -> Total Test time (real) =  40.90 sec
```

The 18 tests comprise 8 Catch2 unit-test executables (46 `TEST\_CASE`s and 85,155 assertions), 2 corpus smoke tests, and 8 independent cross-checks; all pass in this run.

B.2 Tasks 1 and 2

The four word-count configurations were each run three times and the external wall-clock medians are reported. The commands for stop-word derivation, the formal index, the three dictionaries, block construction, and the disk sidecar are as follows.

```
./build/mse count --top 50 --csv .../wordcount-default.csv
./build/mse count --top 50 --stop-words data/stopwords.txt \
    --csv .../wordcount-stop.csv
./build/mse count --top 50 --stem --csv .../wordcount-stem.csv
./build/mse count --top 50 --stop-words data/stopwords.txt --stem \
    --csv .../wordcount-stem-stop.csv
# default: 866626 tokens / 25705 distinct; formal: 556866 / 14937
./build/mse stopwords --derive --min-df-ratio 0.4 --min-entropy-rate 0.93 \
    --out .../derived-df0.40-er0.93.txt --compare data/stopwords.txt
# -> overlap 35 / extra: hath o thee thou / missing: more or than their
./build/mse build --index .../index-formal.bin \
    --stop-words data/stopwords.txt --stem
# -> 195 docs / 14937 terms / 117704 postings / 3685655 bytes / OK
./build/mse build --index .../index-formal-v2.bin \
    --stop-words data/stopwords.txt --stem --compress     # -> 1725694 bytes
./build/mse build --dict splay --index .../index-dict-splay.bin \
    --stop-words data/stopwords.txt --stem                # same for hash/map
./build/mse build-blocks --index .../index-blocks.bin \
    --stop-words data/stopwords.txt --stem
# -> blocks 17 (budget 4096, peak 5129 live)
./build/mse build --index .../formal-v2.bin --dict-file .../formal-v2.bpt \
    --stop-words data/stopwords.txt --stem --compress
./build/mse dump --index .../formal-v2.bin --dict-file .../formal-v2.bpt --verify-all
# -> verified: 14937 terms, 0 mismatches
# -> dict reads: 114 page reads, 29819 cache hits
```

`--verify-all` is implemented only on the `--dict-file` path. A cold query for `banquo` through the sidecar takes 2 page reads and reads 8,308 bytes from the index, 0.225% of a full load.

B.3 Task 3, Threshold Experiments, and Independent Cross-Checks

The query examples and the 201 threshold measurements use the formal index whose processing flags are `stopwords stemmed`. The independent implementations and the RFC corpus

follow; the Python side performs its own tokenization, index parsing, and token-stream scan.

```
./build/mse query --index .../index-formal.bin --stop-words data/stopwords.txt \
    --word osric --pretty        # -> 1 doc; doc 30 "Hamlet"
./build/mse query --index .../index-formal.bin --stop-words data/stopwords.txt \
    --word love                  # -> 138 documents
./build/mse query --index .../index-formal.bin --stop-words data/stopwords.txt \
    --phrase "king richard" --top-x 3        # -> 3 documents
./build/mse query --index .../index-formal.bin --stop-words data/stopwords.txt \
    --words "king queen" --min-match 2 --top-x 5   # -> 5 documents
./build/mse query --index .../index-formal.bin --stop-words data/stopwords.txt \
    --word love --max-df 0.7     # -> 0 documents; note: df ratio above --max-df
python3 tools/threshold_matrix.py --index .../index-formal.bin \
    --stop-words data/stopwords.txt \
    --csv reports/data/csv/tasks/threshold_matrix.csv --svg .../threshold_matrix.svg
# -> csv: ...threshold_matrix.csv  (201 measurements)

python3 tools/check_wordcount.py --manifest data/manifest.tsv --stem \
    --stop-words data/stopwords.txt > /tmp/wordcount-py.csv
cmp /tmp/wordcount-py.csv .../wordcount-stem-stop.csv       # identical
python3 tools/brute_force.py --manifest data/manifest.tsv \
    --index .../formal-v1.bin --stop-words data/stopwords.txt --stem
# -> 14937 terms, result: OK
python3 tools/check_phrases.py --index .../formal-v1.bin \
    --stop-words data/stopwords.txt --stem --queries 300
# -> 305 sampled / 303 checked / 0 mismatches

./build/mse stats --manifest data2/data/manifest.tsv
./build/mse stopwords --derive --manifest data2/data/manifest.tsv \
    --min-df-ratio 0.4 --min-entropy-rate 0.93 \
    --out .../rfc-stopwords.txt --compare data/stopwords.txt   # -> 58 terms
./build/mse build --manifest data2/data/manifest.tsv \
    --stop-words .../rfc-stopwords.txt --stem --index .../rfc-formal.bin
```

B.4 Bonus: B+ Tree Scaling and the 500,000-Document Corpus

The B+ tree experiment uses 4 KiB pages, bulk loading, and 1,000 random lookups that are all verified.

```
for keys in 100000 1000000 10000000 30000000 50000000; do
  ./build/mse bptree-bench --keys "$keys" --lookups 1000 --out .../bpt-"$keys".bpt
done
# 50000000 keys -> height 4 / 184322 pages / 754987008 bytes
#               -> 1000 verified, 4.00 page reads each
# CSV: python3 .../parse_bptree.py -> reports/data/csv/verify/bptree-bench.csv
```

For the 500,000-document synthetic corpus the sequence is generate, count, block-build v1 and v2, and in-memory build; peak memory is the `maximum resident set size` reported by `/usr/bin/time -l`. The corpus and temporary indexes were deleted after collection.

```
python3 tools/gen_synthetic_corpus.py --root /tmp/mse-scale-corpus-500k \
    --docs 500000 --tokens-per-doc 40 --vocab 5000000
# -> 500000 docs / 20000000 tokens / generate 73.67 s
/usr/bin/time -l ./build/mse stats \
    --manifest /tmp/mse-scale-corpus-500k/data/manifest.tsv   # RSS 463.9 MiB
/usr/bin/time -l ./build/mse build-blocks \
    --manifest /tmp/mse-scale-corpus-500k/data/manifest.tsv \
    --block-terms 4096 --index .../scale-500k-v1.bin
# -> blocks 4855 / terms 4908747 / scan 5.47 s + merge 125.08 s
# -> 427603247 bytes / max RSS 1.01 GiB / real 168.16 s
/usr/bin/time -l ./build/mse build \
    --manifest /tmp/mse-scale-corpus-500k/data/manifest.tsv \
    --index .../scale-500k-mem.bin
```

```
# -> 427603247 bytes / max RSS 2.00 GiB / real 76.75 s
# -> cmp with v1: identical (SHA-256 4fcb45...)
```

The archive `reports.zip` organizes the evidence as follows: three result files (`results-tasks.md`, `results-verify.md`, `results-preprocess.md`); `csv/` with the plotting inputs (word frequency, threshold matrix, B+ tree benchmark); `logs/` with 53 raw logs in three groups (30 + 20 + 3); and `work/` with temporary indexes and reproduction scripts, which are not evidence. Timing convention: two analysts measured concurrently on the same machine, so wall-clock times may be raised by parallel load and are meaningful only in order of magnitude. Byte counts, counts, page counts, SHA-256 values, and cross-check results are deterministic.

C Data Format Reference

This appendix summarizes the four data files the engine reads and writes: the corpus manifest, the text layer, the index (v1 and v2), and the disk B+ tree sidecar. The format specification follows the implementation and Sections 2.5.3 and 2.7.

C.1 Corpus Manifest and Text Layer

`manifest.tsv` is tab-separated with a header row and one document per line; the `text` column is resolved relative to the *corpus root* (the engine takes `dirname(dirname(manifest))`). The optional eighth column, `format`, is `marked` (the default, line markers active) or `plain` (the whole line is body text, used by the RFC corpus). The text layer has one unit per line, and a line-initial marker determines the line type; headings and stage directions require a match at both the first and the last character, otherwise the line is body text (Section 1.4.3).

Column / marker	Type	Meaning	Default behavior
`docid` / `kind` / `category`	integer / string	Document ID, work type, and category	—
`title`	string	Work title, used in output and `--pretty`	—
`text` / `source` / `url`	path / string	Text layer, source file path, and source URL	—
`format`	string (optional)	`marked` or `plain`	—
`#` comment	file header	Excluded from statistics	skipped
`[ACT I]` etc.	line starts with `[` and ends with `]`	Heading	not tokenized
`(Exit Ghost)`	line starts with `(` and ends with `)`	Stage direction	not tokenized
`@BERNARDO`	line starts with `@`	Speaker label	not tokenized
Other lines	dialogue, verse, or prose	Body text	tokenized

Note: the three marker classes can be retained independently with `--keep-headings`, `--keep-speakers`, and `--keep-stage`.

C.2 Version 1 Index Layout and Version 2 Differences

All integers are little-endian; a string is a u32 byte length followed by the raw bytes. The v1 index is laid out sequentially as follows.

```
header : magic "MSE1" | version u32 | flags u32 | docCount u32 |
         termCount u32 | tokenCount u64                (28 bytes)
doc    : docID u32 | keptTokens u32 | titleLen u32 | title[...]   (12+len)
term   : termLen u32 | term[...] | df u32 | tfTotal u64 |
         blockOffset u64 | blockBytes u32                (28+len)
block  : df x (docID u32, tf u32), then sum(tf) x position u32
         blockBytes = 8*df + 4*sum(tf)
```

Bit 0 of `flags` records stop-word filtering and bit 1 records stemming; the query side replays the same normalization pipeline from these flags. The term table is strictly increasing, document IDs are dense from 1 to N, document IDs within a posting block strictly increase, and the term frequency equals the number of positions; `validateIndex()` checks these invariants after loading. The v2 index has the same header, document table, and term table as v1 except

that `version` = 2 and each posting block is replaced by "deltas plus byte-aligned variable-length codes." For each posting it writes the document-ID delta (starting from 0 within the block, ≥ 1), the term frequency, and the term-frequency position deltas (starting from 0 within the record, ≥ 0). The first byte's top two bits give the total length and the payload uses 6/14/22/30 bits (`01`/`10`/`11`/`00` for 1/2/3/4 bytes), little-endian.

C.3 Disk B+ Tree Sidecar

The sidecar is a 4 KiB-page file (the page size is configurable). Page 0 is the header page and each later page is one tree node; integers are little-endian. Keys are variable-length strings, the fanout is determined by a byte budget, and leaf pages are linked in order by `nextLeaf`.

```
page 0 (header) : magic "BPT1" | version u32 | pageSize u32 |
                  pageCount u32 | rootId u32 | height u32 |
                  leafCount u64 | keyCount u64
page i+1        : leaf u8 | keyCount u16 | nextLeaf u32 |
                  leaf  : value u64[count] | blob[count]
                  inner : child u32[count+1] | blob[count]
blob            : length u8 | bytes
value           : (posting blockOffset << 21) | blockBytes   (21-bit length)
```

A query uses the B+ tree to locate the posting block without loading the full term table. `validate()` checks invariants I1–I5. The file is written by bulk loading, and it is rebuilt whenever the index is rebuilt.

Author List

Author	Contributions
Cen Ruiyang	Independent correctness reports; the cross-check tools (`check\_wordcount.py`, `brute\_force.py`, and `check\_phrases.py`); the synthetic-corpus generator; plotting scripts; fixes found by the cross-checks; and the revision of this report.
Deng Jun	Project plan and corpus pipeline; tokenizer; word-count statistics; stop-word list; Porter stemmer; dictionary layer; inverted-index format; query program; threshold experiments; and the original report and presentation.
Lin Jiangtao	CMake and Catch2 test infrastructure; entropy-based stop-word derivation; corpus-agnostic importer and the RFC corpus; disk B+ tree; block-based construction; v2 compression; streaming index merge; and code formatting.

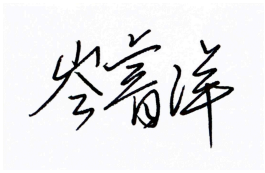
All authors contributed to the integration, testing, and revision of the report.

Declaration

We hereby declare that all the work done in this project titled “Mini Search Engine Project Report” is of our independent effort as a group.

Date: 2026-10-21

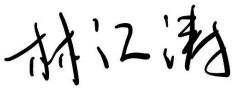
Signatures



Cen Ruiyang



Deng Jun



Lin Jiangtao